# SMAART Minds - Tech Manuals

**Version:** 1.0
**Date:** January 2026
**Platform:** SMAART Minds - Student Dashboard & Learning Management System

---

## 1. Development Environment Setup

### 1.1 Prerequisites

| Software | Minimum Version | Purpose |
|---|---|---|
| Node.js | v14.0+ | Runtime environment |
| npm | v6.0+ | Package manager |
| MongoDB | v4.4+ | Database |
| Git | v2.0+ | Version control |
| VS Code | Latest | Recommended IDE |

### 1.2 Project Structure

```
v.0.1 (04-01-2025)/
├── front-end/                # React frontend application
│   ├── src/
│   │   ├── components/       # Reusable UI components (46+)
│   │   ├── pages/            # Page components (39)
│   │   ├── services/         # API service modules
│   │   ├── contexts/         # React contexts
│   │   ├── hooks/            # Custom React hooks
│   │   ├── utils/            # Utility functions
│   │   ├── assets/           # Static assets
│   │   ├── App.jsx           # Main app component
│   │   ├── main.jsx          # Entry point
│   │   └── index.css         # Global styles
│   ├── public/               # Public static files
│   ├── package.json          # Dependencies
│   └── vite.config.js        # Vite configuration
│
```

```
├── back-end/                # Node.js backend application
│   ├── controllers/         # Request handlers
│   ├── middleware/          # Express middleware
│   ├── models/              # Mongoose schemas (30)
│   ├── routes/              # API routes (32)
│   ├── services/            # Business logic services
│   ├── utils/               # Utility functions
│   ├── scripts/             # Database scripts
│   ├── uploads/             # File upload storage
│   ├── server.js            # Express server entry
│   └── package.json         # Dependencies
│
└── document/                # Documentation files
```

## 1.3 Installation Steps

### Step 1: Clone Repository

```
git clone <repository-url>
cd "v.0.1 (04-01-2025)"
```

### Step 2: Start MongoDB

```
# Windows - Start MongoDB service
mongod

# Or if using MongoDB as service
net start MongoDB
```

### Step 3: Setup Backend

```
cd back-end

# Install dependencies
npm install

# Create .env file
copy .env.example .env
# Edit .env with your configuration

# Start development server
npm run dev
```

### Step 4: Setup Frontend

```
cd front-end

# Install dependencies
npm install

# Start development server
npm run dev
```

### Step 5: Verify Installation

- Backend: http://localhost:5000
- Frontend: http://localhost:5173

---

# 2. Backend Technical Manual

## 2.1 Server Configuration

### File: `server.js`

```
// Key configurations
const PORT = process.env.PORT || 5000;
const MONGODB_URI = process.env.MONGODB_URI;

// Middleware stack
app.use(cors());                    // Cross-origin requests
app.use(express.json());            // JSON body parsing
app.use(express.urlencoded(...));   // URL-encoded bodies
app.use('/uploads', express.static('uploads')); // Static files
```

## 2.2 Database Connection

### Connection String:

```
mongodb://localhost:27017/minds_db
```

### Connection Setup:

```
mongoose.connect(MONGODB_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('✅ MongoDB connected'))
.catch(err => console.error('MongoDB connection error:', err));
```

## 2.3 Model Definitions

### User Model (`models/User.js`)

```
const userSchema = new mongoose.Schema({
  fullName: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  mobileNumber: { type: String, required: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['student', 'admin', 'teacher'], default: 'student' },
  registrationCompleted: { type: Boolean, default: false }
}, { timestamps: true });

// Password hashing pre-save hook
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});
```

### Registration Model (`models/Registration.js`)

```
const registrationSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  email: { type: String, required: true },
  fullName: { type: String, required: true },
  personalDetails: {
    studentId: String,
    dob: Date,
    gender: String,
    address: { doorNo: String, city: String, state: String, country: String, pincode: String
  },
  academicDetails: {
    course: String,
    department: String,
    yearSemester: String,
    rollNumber: String
  },
  marksheets: Object,
```

```
  certificates: Array,
  idProof: Object,
  status: { type: String, enum: ['pending', 'approved', 'rejected'], default: 'pending' }
}, { timestamps: true });
```

## 2.4 Route Configuration

### Base Routes in `server.js`:

```
app.use('/api/auth', require('./routes/auth'));
app.use('/api/users', require('./routes/users'));
app.use('/api/courses', require('./routes/courses'));
app.use('/api/assessments', require('./routes/assessments'));
app.use('/api/results', require('./routes/results'));
app.use('/api/tickets', require('./routes/tickets'));
app.use('/api/community', require('./routes/community'));
app.use('/api/enrollments', require('./routes/enrollments'));
app.use('/api/visionboards', require('./routes/visionBoards'));
```

## 2.5 Middleware Reference

### Authentication Middleware (`middleware/auth.js`)

```
const protect = async (req, res, next) => {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'Not authorized' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    next();
  } catch (err) {
    res.status(401).json({ error: 'Token invalid' });
  }
};

const admin = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required' });
  }
  next();
};
```

### File Upload Middleware (`middleware/upload.js`)

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => cb(null, Date.now() + path.extname(file.originalname))
});

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
  fileFilter: (req, file, cb) => {
    const allowed = ['image/jpeg', 'image/png', 'application/pdf'];
    cb(null, allowed.includes(file.mimetype));
  }
});
```

# 3. Frontend Technical Manual

## 3.1 Application Configuration

### Entry Point: `main.jsx`

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

### Main App Component: `App.jsx`

```
const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <SidebarProvider>
        <Toaster />
        <Sonner />
        <BrowserRouter>
          <AnimatedRoutes />
          <FloatingCommunityButton />
          <SecurityGuard />
        </BrowserRouter>
      </SidebarProvider>
    </TooltipProvider>
```

```
  </QueryClientProvider>
);
```

## 3.2 Routing Configuration

**File:** `components/AnimatedRoutes.jsx`

Key Routes:

```
// Public routes
<Route path="/" element={<Institution />} />
<Route path="/landing" element={<LandingPage />} />

// Protected routes (require authentication)
<Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>}>
  <Route index element={<DashboardHome />} />
  <Route path="profile" element={<Profile />} />
  <Route path="courses" element={<MyCourses />} />
  <Route path="assessments" element={<MyAssessments />} />
  <Route path="community" element={<Community />} />
  <Route path="settings" element={<Settings />} />
  <Route path="help" element={<Help />} />
</Route>

// Assessment routes
<Route path="/baseline-test" element={<BaseLineTest />} />
<Route path="/eq-test" element={<EQTest />} />
<Route path="/aiq-test" element={<AIQTest />} />
// ... other assessments
```

## 3.3 API Service Configuration

**File:** `services/api.js`

```
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000';

// Get authentication headers
const getAuthHeaders = () => {
  const token = sessionStorage.getItem('token');
  return {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json'
  };
};

// Example API function
export const fetchUserData = async (userId) => {
```

```
  const response = await fetch(`${API_BASE_URL}/api/users/${userId}`, {
    headers: getAuthHeaders()
  });
  return response.json();
};
```

## 3.4 State Management

### Session Storage (Authentication)

```
// Store after login
sessionStorage.setItem('token', jwtToken);
sessionStorage.setItem('user', JSON.stringify(userData));

// Retrieve for API calls
const token = sessionStorage.getItem('token');

// Clear on logout
sessionStorage.clear();
```

### React Query (Server State)

```
import { useQuery, useMutation } from '@tanstack/react-query';

// Fetch data with caching
const { data, isLoading, error } = useQuery({
  queryKey: ['user', userId],
  queryFn: () => fetchUserData(userId)
});

// Mutations
const mutation = useMutation({
  mutationFn: updateUser,
  onSuccess: () => queryClient.invalidateQueries(['user'])
});
```

## 3.5 Component Patterns

### Page Component Template

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
```

```
const PageComponent = () => {
  const navigate = useNavigate();
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchData();
  }, []);

  const fetchData = async () => {
    try {
      const response = await apiCall();
      setData(response);
    } catch (error) {
      console.error('Error:', error);
    } finally {
      setLoading(false);
    }
  };

  if (loading) return <LoadingSpinner />;

  return (
    <div className="page-container">
      {/* Page content */}
    </div>
  );
};

export default PageComponent;
```

# 4. Database Administration

## 4.1 MongoDB Commands

### Connect to Database

```
mongosh
use minds_db
```

### View Collections

```
show collections
```

### Query Examples

```
// Find all users
db.users.find()

// Find user by email
db.users.findOne({ email: "test@example.com" })

// Find all registrations
db.registrations.find()

// Count documents
db.users.countDocuments()

// Find with projection
db.users.find({}, { email: 1, fullName: 1 })
```

## 4.2 Database Indexes

```
// Users collection
db.users.createIndex({ email: 1 }, { unique: true });
db.users.createIndex({ role: 1 });

// Assessments
db.results.createIndex({ userId: 1 });
db.results.createIndex({ assessmentId: 1 });
db.results.createIndex({ userId: 1, assessmentId: 1 });

// Support tickets
db.support.createIndex({ ticketId: 1 }, { unique: true, sparse: true });
db.support.createIndex({ userId: 1 });
db.support.createIndex({ status: 1, priority: 1 });
```

## 4.3 Backup & Restore

```
# Backup database
mongodump --db minds_db --out /backup/

# Restore database
mongorestore --db minds_db /backup/minds_db/
```

# 5. API Reference

## 5.1 Authentication APIs

### Login

```
POST /api/auth/login
Content-Type: application/json

Request:
{ "email": "user@example.com" }

Response:
{ "message": "OTP sent", "success": true }
```

### Verify OTP

```
POST /api/auth/verify-otp
Content-Type: application/json

Request:
{ "email": "user@example.com", "otp": "123456" }

Response:
{
  "success": true,
  "token": "jwt-token-here",
  "user": { "id": "...", "email": "...", "name": "..." }
}
```

## 5.2 User APIs

### Get User Profile

```
GET /api/users/:id
Authorization: Bearer <token>

Response:
{
  "_id": "ObjectId",
  "fullName": "John Doe",
  "email": "john@example.com",
  "role": "student",
  "registrationCompleted": true
}
```

### Save Registration

```
POST /api/users/register-details
Content-Type: application/json
Authorization: Bearer <token>

Request:
{
  "email": "user@example.com",
  "fullName": "User Name",
  "mobileNumber": "9876543210",
  "password": "securepassword",
  "personalDetails": {...},
  "academicDetails": {...}
}

Response:
{ "message": "Registration saved successfully" }
```

## 5.3 Assessment APIs

### Start Assessment

```
GET /api/assessments/:id/start
Authorization: Bearer <token>

Response:
{
  "resultId": "ObjectId",
  "questions": [...],
  "responses": [...]
}
```

### Save Answer

```
POST /api/results/save-answer
Authorization: Bearer <token>

Request:
{
  "resultId": "ObjectId",
  "questionId": "ObjectId",
  "selectedValue": "A"
}

Response:
{ "success": true }
```

### 5.4 Course APIs

#### List Courses

```
GET /api/courses
Authorization: Bearer <token>

Response:
{
  "courses": [
    { "_id": "...", "title": "...", "modules": [...] }
  ]
}
```

#### Enroll in Course

```
POST /api/courseenrollments
Authorization: Bearer <token>

Request:
{ "courseId": "ObjectId" }

Response:
{ "message": "Enrolled successfully", "enrollment": {...} }
```

---

## 6. Design System Reference

### 6.1 Color Palette

```
/* Primary Colors */
--deep-blue: #004D99;       /* HSL: 210 95% 30% */
--teal: #42A89B;            /* HSL: 170 50% 45% */

/* Accent Colors */
--warm-gold: #B58539;       /* HSL: 40 50% 55% */
--light-gold: #EBCC5C;      /* HSL: 48 100% 50% */

/* Neutral Colors */
--background: hsl(0, 0%, 98%);
--foreground: hsl(210, 30%, 20%);
--card: hsl(0, 0%, 100%);
--border: hsl(0, 0%, 90%);
```

### 6.2 Typography

```
/* Font Families */
--font-primary: 'Poppins', sans-serif;
--font-secondary: 'Nunito', sans-serif;

/* Font Sizes */
--text-xs: 0.75rem;   /* 12px */
--text-sm: 0.875rem;  /* 14px */
--text-base: 1rem;    /* 16px */
--text-lg: 1.125rem;  /* 18px */
--text-xl: 1.25rem;   /* 20px */
--text-2xl: 1.5rem;   /* 24px */
--text-3xl: 1.875rem; /* 30px */
--text-4xl: 2.25rem;  /* 36px */
```

### 6.3 Component Classes

```
/* Buttons */
.btn-primary { /* Deep Blue gradient */ }
.btn-secondary { /* Teal gradient */ }
.btn-accent { /* Warm Gold gradient */ }

/* Cards */
.card-premium { /* White bg, soft shadow, rounded */ }
.glass-effect { /* Frosted glass appearance */ }

/* Effects */
.hover-lift { /* Elevation on hover */ }
.text-gradient-blue { /* Blue gradient text */ }
```

---

## 7. Deployment Guide

### 7.1 Production Environment

#### Environment Variables

```
# Backend .env (production)
NODE_ENV=production
PORT=5000
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/minds
JWT_SECRET=your-strong-secret-key
```

```
CLOUDINARY_CLOUD_NAME=your-cloud
CLOUDINARY_API_KEY=your-key
CLOUDINARY_API_SECRET=your-secret
```

### Build Commands

```
# Frontend build
cd front-end
npm run build

# Backend (no build needed, just start)
cd back-end
npm start
```

## 7.2 Server Requirements

| Resource | Minimum | Recommended |
|---|---|---|
| CPU | 1 core | 2+ cores |
| RAM | 2GB | 4GB+ |
| Storage | 10GB | 20GB+ |
| Bandwidth | 10Mbps | 100Mbps+ |

## 7.3 Security Checklist

- [ ] Use HTTPS in production
- [ ] Set secure CORS origins
- [ ] Enable rate limiting
- [ ] Use strong JWT secrets
- [ ] Enable MongoDB authentication
- [ ] Set up firewall rules
- [ ] Configure log rotation
- [ ] Set up monitoring

---

**Document End**

*This technical manual provides development and deployment guidance for the SMAART Minds platform.*